Objects and Classes

OAVS Computer Teacher Competitive Exam — Revision Notes (Java)

Contents

1. Object Fundamentals
2. Pass by Value
3. 'this' Reference
4. Data Hiding and Encapsulation
5. Overloading
6. Overriding
7. Constructors
8. Finalization
9. Subclasses (Inheritance)
10. Relationship between super class object and subclass object
11. Implicit subclass object to super class object Conversion
12. Dynamic method dispatch

1 Object Fundamentals

Definition & Core Concept

A **class** is a blueprint/template that defines the **state** (fields/variables) and **behavior** (methods) common to a group of objects. An **object** is a **runtime instance** of a class, created using the `new` keyword. Every object has three defining characteristics:

- **State** – the data/values held in the object's instance variables at a given moment.
- **Behavior** – the operations (methods) that can act on or change the state.
- **Identity** – a unique existence in memory that distinguishes it from every other object, even one with identical state.

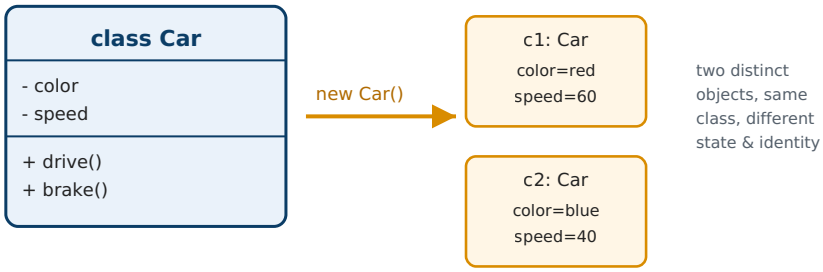


Fig 1: One class, Car, used as a blueprint to create two independent objects c1 and c2.

Example – State, Behavior & Identity Together

```
class Student {
    String name;   // instance variable -> part of STATE
    int marks;     // instance variable -> part of STATE

    void updateMarks(int m) {   // method -> BEHAVIOR
        marks = m;
    }
}
class Test {
    public static void main(String[] args) {
        Student s1 = new Student();   // object 1 created
        s1.name = "Aman"; s1.marks = 80;

        Student s2 = new Student();   // object 2 created
        s2.name = "Aman"; s2.marks = 80;

        System.out.println(s1 == s2);        // false
        System.out.println(s1.equals(s2));   // false (default Object.equals)

        s1.updateMarks(95);   // behavior changes s1's state
        System.out.println(s1.marks);  // 95
        System.out.println(s2.marks);  // 80 (unaffected)
    }
}
```

Output:
false
false
95
80

How this maps to the three concepts: **State** – `s1` and `s2` start with identical values (`name="Aman"`, `marks=80`), the data held in their instance variables at that moment. **Behavior** – calling `s1.updateMarks(95)` is behavior acting on `s1`'s own state and changing it; `s2` stays untouched because it is a separate object. **Identity** – even though `s1` and `s2` hold exactly the same values, `s1 == s2` is `false`, since they occupy different memory addresses on the heap. That is identity – it exists independently of what the state happens to be.

> **EXAM POINT**
> Two objects with identical state are still **not** the same object — identity is about the reference/address, not the field values. The default (unoverridden) `equals()` in class `Object` simply performs a `==` comparison, so it also returns `false` here.

Object Creation Syntax

```
ClassName refVariable = new ClassName(arguments);
// example
Car c1 = new Car();
```

The `new` operator allocates memory for the object on the **heap** and returns a reference, which is stored in the reference variable (on the stack for local variables).

Working Mechanism

1. JVM allocates memory for the object on the heap based on the class definition.
2. Instance variables are given default values (0, false, null).
3. The constructor is invoked to initialize the object's state.

4. A reference (address) to the newly created object is returned and assigned to the reference variable.

Step-by-step walkthrough for `Car c1 = new Car();`

Step	What happens	State after this step
1	JVM allocates heap memory sized to hold a `Car` object (space for `color` and `speed`).	Unnamed block of heap memory exists; not yet usable.
2	Fields are auto-initialized to Java's default values before any constructor code runs.	`color = null`, `speed = 0`
3	The (compiler-supplied) no-arg constructor of `Car` executes. Since `Car` defines no explicit constructor here, this step does nothing extra to the defaults.	`color = null`, `speed = 0` (unchanged)
4	The address of this fully-built object is returned by `new` and stored in reference variable `c1`.	`c1` now points to the object; `c1.color` and `c1.speed` are accessible.

Example

```
class Car {
    String color;
    int speed;
    void display() {
        System.out.println(color + " " + speed);
    }
}
class Test {
    public static void main(String[] args) {
        Car c1 = new Car();  // steps 1-4 happen here
        c1.color = "Red"; c1.speed = 60;  // AFTER construction: assign fields
        Car c2 = new Car();  // steps 1-4 repeat for a new, separate object
        c2.color = "Blue"; c2.speed = 40;
        c1.display();
        c2.display();
    }
}
```

Output:
Red 60
Blue 40

EXAM POINT

Object identity is checked using == on references (compares memory address), while state equality is checked using `.equals()`. Two objects with same state are still different objects (different identity).

```
Car a = new Car(); a.color="Red"; a.speed=60;
Car b = new Car(); b.color="Red"; b.speed=60;  // same state as a
Car c = a;  // c now refers to the SAME object as a

System.out.println(a == b);        // false -> different objects (different identity)
System.out.println(a == c);        // true  -> same object (same identity)
System.out.println(a.equals(b));   // false -> default equals() just does == internally
```

Output:
false
true
false

`a` and `b` hold identical state but are two separate objects on the heap, so `==` gives `false`. `c` was assigned `c = a`, so it points to the **same** object as `a` — hence `a == c` is `true`. Since `Car` does not override `equals()`, it falls back to `Object`'s default behavior, which is identical to ==, so `a.equals(b)` is also `false` even though their fields match.

COMMON CONFUSION

A class itself occupies no memory for instance data until an object is created; only `static` members belong to the class and exist without any object.

SUMMARY

Class = blueprint; Object = instance with state, behavior, identity; created using `new`; stored on heap; reference stored in a variable.

2 Pass by Value

Definition & Core Concept

Java is strictly pass-by-value. When a method is called, a **copy** of the argument is passed — never the original variable itself. This applies uniformly to both primitives and object references:

- For **primitives**, the actual value (e.g., an int) is copied. Changes inside the method do not affect the caller's variable.
- For **objects**, the **value of the reference** (i.e., the memory address) is copied — not the object itself. Both the original and copied reference point to the **same object** on the heap, so changes made *through* the reference (modifying fields) are visible to the caller, but **reassigning** the copied reference inside the method does not affect the caller's original reference.

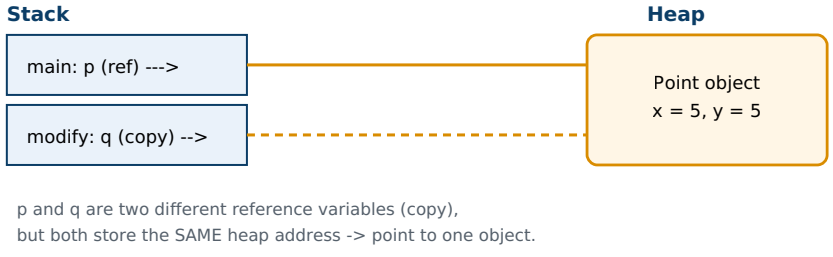


Fig 2: Reference value is copied; both refs point to the same object on the heap.

Example 1 – Primitive (no change reflected)

```
void change(int n) {
    n = 99;              // only changes the LOCAL copy n
}
int x = 5;              // x = 5
change(x);               // value 5 is COPIED into n; x itself is untouched
System.out.println(x);   // x is still 5
```

Output: 5

Why: calling `change(x)` copies the value 5 into the parameter `n`. Inside the method, `n` is a completely separate local variable — setting `n = 99` only changes that copy. The original `x` in `main` is untouched, so it still prints 5.

Example 2 – Object field modified through reference (visible)

```
class Point { int x, y; }
void modify(Point q) {
    q.x = 100;                // changes the FIELD of the shared object
}
Point p = new Point();  // p points to a new Point object
p.x = 5;                        // p.x = 5
modify(p);                      // reference VALUE copied into q; q and p -> SAME object
System.out.println(p.x);        // prints 100 - object was modified through q
```

Output: 100 (field changed via same object)

Why: `modify(p)` copies the **reference value** (heap address) held in `p` into the parameter `q`. Both `p` and `q` are now two different variables, but they point to the **same** `Point` object on the heap. So `q.x = 100` changes a field on that shared object, and the change is visible through `p` too — hence the output is `100`, not 5.

Example 3 – Reassigning reference inside method (NOT visible)

```
void reassign(Point q) {
    q = new Point();   // q now points to a NEW, different object
    q.x = 500;                // only the NEW object's field is set
}
Point p = new Point();  // p points to the original object
p.x = 5;                        // p.x = 5
reassign(p);                    // q's copy is redirected; p in main is unaffected
System.out.println(p.x);        // prints 5 - p still points to the original object
```

Output: 5 (p in main still points to the original object)

Why: inside `reassign`, the statement `q = new Point();` makes the **local copy** `q` point to a **brand-new** object, abandoning the original one. `q.x = 500` only sets a field on this new object. Since `p` in `main` was never touched — only q's copy of the reference was redirected — `p` still points to the original object with `x = 5`. This is exactly why Java is not pass-by-reference: reassigning the parameter never reassigns the caller's variable.

> **EXAM POINT**
> Java has **no pass-by-reference**. Frequently asked as a trick MCQ: "Java supports pass by reference for objects" – this statement is **FALSE**. It is pass-by-value of the reference.

COMMON CONFUSION

Do not confuse "reference is copied" with "pass by reference." A true pass-by-reference would let the callee change what the caller's variable points to (as in Example 3) — which Java does NOT allow.

SUMMARY

Java = pass-by-value always. Primitives: value copied. Objects: reference value (address) copied → shared object can be mutated, but reference reassignment is local to the method.

3 'this' Reference

Definition & Core Concept

`this` is an implicit reference variable available inside every non-static (instance) method and constructor, which refers to the **current object** — the object on which the method was invoked. It is automatically passed by the JVM; the programmer never needs to declare it.

Common Uses of `this`

- **Resolving naming conflicts** between instance variables and parameters/local variables of the same name.
- **Invoking current class constructor** from another constructor – `this(...)` (constructor chaining).
- **Passing the current object** as an argument to another method.
- **Returning the current object** from a method (used for method chaining).
- **Invoking current class methods explicitly** for clarity (rarely needed).

Example – Resolving Shadowing

```
class Student {
    String name;
    Student(String name) {
        this.name = name; // this.name = instance var, name = parameter
    }
}
```

Example – Constructor Chaining with `this()`

```
class Box {
    int l, b, h;
    Box() { this(1, 1, 1); // calls parameterized constructor
        System.out.println("default ctor"); }
    Box(int l, int b, int h) {
        this.l=l; this.b=b; this.h=h; }
}
Box b = new Box();
System.out.println(b.l);
```

Output:

default ctor
1

EXAM POINT

`this()` call, if used, **must be the first statement** in a constructor. A constructor cannot have both `this(...)` and `super(...)` together (only one, and it must be first).

COMMON CONFUSION

`this` cannot be used in a `static` context/method, because static methods belong to the class, not to any specific object.

SUMMARY

`this` = reference to the current object; resolves naming conflicts, enables constructor chaining (`this()` must be first statement), supports method chaining, unusable in static context.

4 Data Hiding and Encapsulation

Definition & Core Concept

Encapsulation is the OOP mechanism of binding data (fields) and the methods that operate on that data into a single unit (class), while restricting direct access to the internal state from outside. **Data hiding** is the specific technique used to achieve this: declaring fields as `private` so they cannot be accessed directly from outside the class, and exposing controlled access via `public` getter/setter methods.

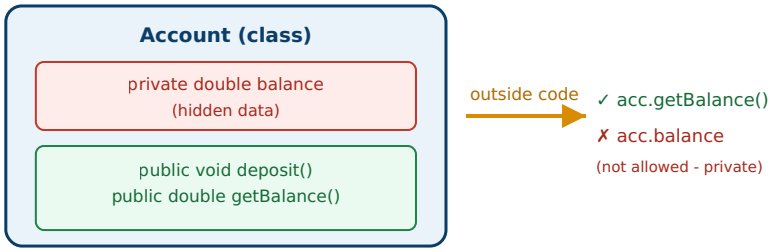


Fig 3: private field hidden from outside; access only via public methods.

Example

```
class Account {
    private double balance;
    public void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
    public double getBalance() { return balance; }
}
// Test
Account acc = new Account();
acc.deposit(500);
System.out.println(acc.getBalance());
// acc.balance = -1000; // COMPILE ERROR: balance has private access
```

Output: 500.0

EXAM POINT

Encapsulation increases **modularity, security, and maintainability** — internal implementation can change without affecting outside code that only uses the public interface. It is achieved primarily through `private` fields + `public` getters/setters.

COMMON CONFUSION

Data hiding and encapsulation are related but not identical: encapsulation = wrapping data+methods together (bundling); data hiding = restricting visibility of that data. Data hiding is one benefit/aspect achieved through encapsulation.

SUMMARY

Encapsulation binds data and methods together; data hiding restricts field access using `private`; controlled access given via public getters/setters; improves security and maintainability.

5 Overloading

Definition & Core Concept

Method Overloading is defining multiple methods in the **same class** (or between a superclass and subclass) with the **same name** but a **different parameter list** (different number, type, or order of parameters). It is an example of **compile-time (static) polymorphism**, resolved by the compiler based on the method signature at compile time.

Rules

- Method name must be the same; parameter list must differ.
- Return type alone **cannot** distinguish overloaded methods.
- Access modifiers and `throws` clause can differ freely.
- Constructors can also be overloaded (constructor overloading).

Example

```
class Calc {
    int add(int a, int b) { return a+b; }
    double add(double a, double b) { return a+b; }
    int add(int a, int b, int c) { return a+b+c; }
}
Calc c = new Calc();
System.out.println(c.add(2,3));
System.out.println(c.add(2.5,3.5));
System.out.println(c.add(1,2,3));
```

Output:

5
6.0
6

6 Overriding

Definition & Core Concept

Method Overriding occurs when a **subclass** provides its **own implementation** of a method that is already defined in its **superclass**, with the **same name, same parameter list, and compatible return type**. It is an example of **runtime (dynamic) polymorphism**, resolved at runtime based on the actual object type.

Rules

- Method signature (name + parameters) must be identical in super and subclass.
- Return type must be same, or a **covariant type** (subtype of original return type).
- Access modifier in subclass cannot be more restrictive than in superclass.
- Overriding method cannot throw broader checked exceptions than the overridden method.
- `static`, `final`, and `private` methods **cannot** be overridden (static methods can only be hidden/redeclared).
- `@Override` annotation is recommended (not mandatory) to let the compiler verify correctness.

Example

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}
Animal a = new Dog();
a.sound();
```

Output: Dog barks

Overloading vs Overriding

Aspect	Overloading	Overriding
Polymorphism type	Compile-time (static)	Runtime (dynamic)
Where defined	Same class (or subclass with diff signature)	Superclass and subclass
Parameter list	Must differ	Must be identical
Return type	Can differ freely	Same or covariant only
Binding	Resolved by compiler	Resolved by JVM at runtime
Inheritance needed?	No	Yes (IS-A relationship)
`private`/`static`	Allowed	Not overridable

EXAM POINT

Overriding is the mechanism behind **dynamic method dispatch**; overloading is decided purely by the **reference/argument types visible to the compiler**. A very common exam question gives code with both and asks the output – always check which object type is actually being referenced at runtime for overridden calls.

COMMON CONFUSION

Changing only the return type (keeping name and parameters same) is neither valid overloading nor overriding — it causes a **compile error** ("method already defined").

SUMMARY

Overloading = same name, different parameters, compile-time; Overriding = same signature in subclass, runtime, needs inheritance and IS-A relationship.

7 Constructors

Definition & Purpose

A **constructor** is a special block of code, syntactically resembling *(similar to)* a method, that is automatically invoked when an object is created using new. Its purpose is to **initialize the state** (instance variables) of the newly created object.

Rules

- Constructor name must exactly match the class name.
- A constructor has **no return type**, not even `void`.

- If no constructor is written, the compiler automatically supplies a **default (no-arg) constructor**.
- The moment you define **any** constructor, the compiler-supplied default constructor is **no longer added**.
- Constructors can be **overloaded** (constructor overloading, see Topic 5).
- Constructors are **not inherited** by subclasses, but a subclass constructor implicitly (or explicitly via `super()`) calls a superclass constructor.
- Cannot be declared `static`, `final`, or `abstract`.

Types Relevant to the Syllabus

Type	Description
Default constructor	Auto-supplied by compiler when no constructor is written; initializes fields to default values.
No-argument constructor	Explicitly written by programmer, takes no parameters.
Parameterized constructor	Accepts arguments to initialize fields with specific values.

Constructor Execution Flow (Object Creation)

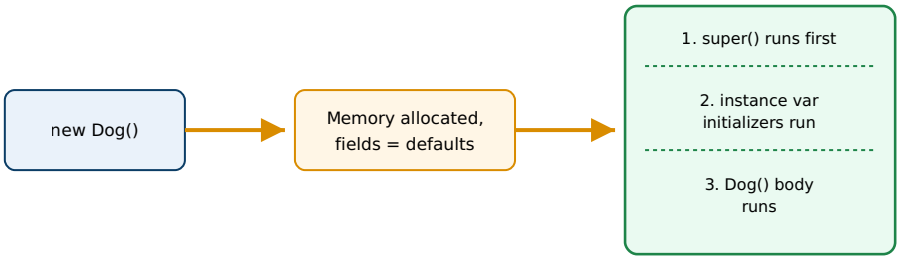


Fig 4: Object creation sequence – allocation, default field values, implicit/explicit super() call, then constructor body.

Example

```
class Point {
    int x, y;
    Point() { this(0,0); }              // no-arg
    Point(int x, int y) { this.x=x; this.y=y; } // parameterized
}
Point p1 = new Point();
Point p2 = new Point(3,4);
System.out.println(p1.x + "," + p1.y);
System.out.println(p2.x + "," + p2.y);
```

Output:
0,0
3,4

EXAM POINT

Every constructor's first statement is (implicitly or explicitly) a call to a superclass constructor via `super()`; if not written, the compiler inserts a no-arg `super()` call automatically. If the superclass has no no-arg constructor, the subclass **must** explicitly call `super(args)`, else compile error.

COMMON CONFUSION

Constructors are NOT methods — they cannot be called directly like `obj.Point()`, cannot be inherited, and are not overridden (only overloaded).

SUMMARY

Constructor initializes object state, same name as class, no return type; default constructor auto-added only if none defined; can be overloaded; first line implicitly/explicitly calls `super()`.

8 Finalization

Definition & Core Concept

Finalization is the process, opposite of construction, by which an object can perform **clean-up actions** (such as releasing non-Java resources: file handles, sockets, native resources) just before it is **garbage collected** by the JVM. In Java, this was traditionally implemented by overriding the `protected void finalize()` method inherited from the `Object` class.

Working Mechanism

- When an object becomes **unreachable** (no live references point to it), it becomes eligible for garbage collection (GC).
- Before the memory is reclaimed, the JVM **may** call the object's `finalize()` method exactly once, to allow clean-up.
- After `finalize()` executes, the object's memory is reclaimed by the garbage collector.

Syntax

```
class Sample {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("object being finalized");
    }
}
```

EXAM POINT

The **timing** of finalize() call is **not guaranteed** — it depends entirely on when (and if) the garbage collector runs; it may never run if the JVM exits first. Calling `System.gc()` only *requests* garbage collection, it does not force it. `finalize()` has been **deprecated** in modern Java (since Java 9) in favor of try-with-resources / `Cleaner`, but remains an important theory topic for exams.

COMMON CONFUSION

Finalization is **not** the same as manually deallocating memory (as in C++ destructors called deterministically). Java's finalize() is not deterministic and should never be relied upon for critical resource release.

SUMMARY

finalize() = clean-up method called by GC before reclaiming an unreachable object's memory; timing not guaranteed; inherited from Object class; deprecated in modern Java but exam-relevant.

9 Subclasses (Inheritance)

Definition & Core Concept

Inheritance is the mechanism by which a new class (**subclass**/derived/child class) acquires the fields and methods of an existing class (**superclass**/base/parent class), using the `extends` keyword. It establishes an **IS-A relationship** and supports code reusability and hierarchical classification.

Syntax

```
class Superclass { ... }
class Subclass extends Superclass { ... }
```

Key Rules

- Java supports only **single inheritance** for classes (a class can extend only one superclass directly).
- A subclass inherits all `public` and `protected` members; `private` members exist in the object but are not directly accessible in the subclass.
- A subclass can add new fields/methods and can override inherited methods.
- Every class in Java implicitly extends `Object` if no other superclass is specified.

Example

```
class Animal {
    String name;
    void eat() { System.out.println(name + " eats"); }
}
class Dog extends Animal {
    void bark() { System.out.println(name + " barks"); }
}
Dog d = new Dog();
d.name = "Tommy";
d.eat();   // inherited method
d.bark();  // own method
```

Output:
Tommy eats
Tommy barks

EXAM POINT

Constructors and `private` members are **not inherited**. Multiple inheritance of classes is **not supported** in Java (to avoid the Diamond Problem) — only achievable via interfaces.

COMMON CONFUSION

"Inherited" does not mean "copied" — the subclass object simply has access to superclass members as part of one combined object; there is only one object in memory even though it has a two-level (or multi-level) class hierarchy.

SUMMARY

Inheritance: subclass extends superclass, gains public/protected members, IS-A relationship, single inheritance only for classes, constructors not inherited.

10 Relationship between super class object and subclass object

Core Concept

When a subclass object is created, it is **not two separate objects** glued together — it is **one single object** in memory that contains **both** the superclass part (inherited fields) and the subclass-specific part (its own fields), as one combined structure on the heap. Conceptually, every subclass object **"contains" the complete superclass object within it** as its base portion.

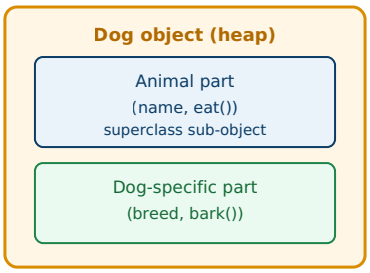


Fig 5: A single Dog object physically contains its Animal (superclass) sub-object as its base part.

Key Points

- A subclass object **"is-a"** superclass object — it can always be used wherever a superclass object is expected.
- A **superclass object cannot**, however, be treated as a subclass object, since it lacks the subclass-specific extensions (no such guarantee of extra members).
- Only **one object** exists on the heap per new call; superclass and subclass "parts" are not separate heap objects.

EXAM POINT

The relationship is asymmetric: *every Dog is an Animal, but not every Animal is a Dog.* This asymmetry is the foundation for both implicit upcasting (Topic 11) and dynamic method dispatch (Topic 12).

SUMMARY

Subclass object = one combined object containing the superclass portion as its base plus its own extensions; IS-A relationship is directional (subclass → superclass only).

11 Implicit subclass object to super class object Conversion

Definition & Core Concept

Because a subclass object always contains a complete superclass sub-object within it (Topic 10), Java allows a subclass object's reference to be assigned to a superclass reference variable **automatically**, without any explicit cast. This is called **upcasting** or **widening reference conversion**, and it is always **type-safe** since the subclass object guarantees at least everything the superclass has.

Syntax

```
Superclass ref = new Subclass();  // implicit, no cast needed
```

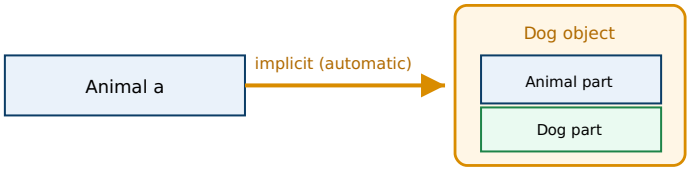


Fig 6: Superclass reference "a" points to a Dog object – only the Animal part is visible through it.

Example 1 – Upcasting (Implicit)

```
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal {
    void bark() { System.out.println("barking"); }
}
Animal a = new Dog();  // implicit upcast
a.eat();       // OK - defined in Animal
// a.bark();   // COMPILE ERROR - bark() not visible via Animal reference
```

Output: eating

Example 2 – Downcasting (Explicit)

```
Animal a = new Dog();      // upcast (implicit) - actual object IS a Dog
Dog d = (Dog) a;            // downcast (explicit) - safe here, works fine
d.bark();                     // OK - Dog's own method now accessible

Animal a2 = new Animal();    // actual object is a plain Animal, NOT a Dog
Dog d2 = (Dog) a2;          // compiles fine, but throws at RUNTIME
d2.bark();                    // ClassCastException - a2 is not really a Dog
```

Output:
barking
Exception in thread "main" java.lang.ClassCastException: class Animal cannot be cast to class Dog

Why: a cast like `(Dog)` a only **compiles** if `Dog` and `Animal` are related by inheritance — the compiler allows it optimistically. But whether it actually **succeeds** depends on the **real object** on the heap at runtime. In the first case, a genuinely points to a `Dog`, so the cast succeeds. In the second case, `a2` points to a plain `Animal` object that was never a `Dog`, so the JVM throws `ClassCastException` the moment the cast is executed.

EXAM POINT

Upcasting (subclass→superclass) is **implicit**; the reverse, downcasting (superclass→subclass), always needs an **explicit cast** and can throw `ClassCastException` at runtime if the actual object is not truly of that subclass type. The compiler decides **which members are accessible** based on the **reference type**, not the actual object type — but **which overridden method version runs** is decided at runtime based on the **actual object type** (see Topic 12).

COMMON CONFUSION

Upcasting does not lose or change the object — the object on the heap remains a full Dog object; only the reference's "view" is restricted to what Animal declares. Overridden methods (not new subclass-only methods) still execute their Dog version at runtime.

SUMMARY

Implicit conversion = subclass reference automatically assignable to superclass reference (upcasting); always safe; reverse (downcasting) needs explicit cast and may fail at runtime.

12 Dynamic method dispatch

Definition & Core Concept

Dynamic method dispatch is the mechanism by which a call to an **overridden method** is resolved **at runtime**, based on the **actual type of the object** being referred to, rather than the type of the reference

variable used to call it. This is how Java implements **runtime polymorphism**, and it works through implicit upcasting (Topic 11) combined with overriding (Topic 6).

Step-by-Step Working Mechanism

1. A superclass reference variable is used to point to a subclass object (implicit upcast).
2. At **compile time**, the compiler only checks that the method exists in the **reference type** (superclass) — it does not decide which version to run.
3. At **runtime**, the JVM looks at the **actual object** on the heap that the reference points to.
4. The JVM invokes the **overridden version defined in the actual object's class** (the most specific overriding version in the runtime class or its nearest ancestor that overrides it).

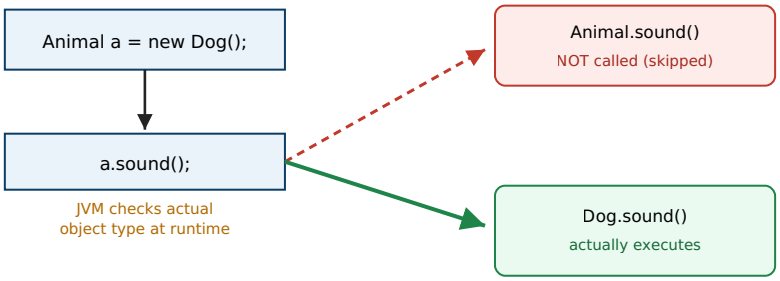


Fig 7: Reference type is Animal, but runtime object is Dog → Dog's overridden sound() actually runs (dynamic binding).

Example

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Cat meows"); }
}
class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.sound();          // Dog's version runs
        a = new Cat();
        a.sound();          // Cat's version runs
    }
}
```

Output:
Dog barks
Cat meows

EXAM POINT

Dynamic dispatch applies only to **overridden instance methods**. It does **not** apply to: `static` methods (resolved by reference type – "method hiding", not overriding), `private` methods, `final` methods (cannot be overridden at all), or instance/field access (fields are always resolved using the **reference type**, at compile time – fields are *not* dynamically dispatched).

COMMON CONFUSION

A very common exam trap: accessing a **field** (not a method) through a superclass reference always gives the superclass's field, even if the object is a subclass instance with a field of the same name (field hiding, resolved statically) — unlike overridden methods, which resolve dynamically.

SUMMARY

Dynamic method dispatch = runtime resolution of overridden method calls based on actual object type, not reference type; enables runtime polymorphism; applies only to overridable instance methods, not to fields or static/private/final methods.

Quick Revision Section

Key Definitions (One-Liners)

Term	One-line Definition
Object	Runtime instance of a class with state, behavior, identity
Pass by value	Copy of value (or reference value) passed to a method; Java has no pass-by-reference
`this`	Implicit reference to the current object
Encapsulation	Bundling data + methods; restricting access via private fields
Overloading	Same method name, different parameters, compile-time
Overriding	Same signature redefined in subclass, runtime
Constructor	Special block, same name as class, no return type, initializes object
Finalization	finalize() called by GC before reclaiming unreachable object
Inheritance	Subclass acquires superclass members via extends; IS-A relation
Upcasting	Implicit assignment of subclass object to superclass reference
Dynamic dispatch	Overridden method resolved at runtime by actual object type

Important Rules to Remember

- Java = always pass-by-value (never pass-by-reference), even for objects.
- `this()` / `super()` must be the **first statement** in a constructor, and only one of them can be used.
- Default constructor is auto-added **only if** no constructor is explicitly defined.
- Constructors: not inherited, not overridden, only overloaded.
- `static`, `private`, `final` methods → cannot be overridden (no dynamic dispatch).
- Upcasting (subclass→super) = implicit; downcasting (super→subclass) = explicit cast, risk of `ClassCastException`.
- Field access is always resolved statically (by reference type); overridden method calls are resolved dynamically (by object type).
- Java supports single inheritance only for classes (no multiple class inheritance).

Overloading vs Overriding (Recap Table)

Point	Overloading	Overriding
Binding	Static (compile-time)	Dynamic (runtime)

Parameters	Different	Same
Class relation	Not required	Superclass-subclass required

Likely Exam Traps

- "Java supports pass-by-reference" → FALSE, it is pass-by-value of the reference.
- Field hiding vs method overriding → fields use reference type; overridden methods use object type.
- Return-type-only difference → invalid overload, compile error.
- finalize() call timing → not guaranteed, JVM-dependent.
- Superclass reference calling a subclass-only method without cast → compile error.